Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Judul Paper IEEE Access!

YASIN ARIL MUSTOFA[1,3], (Member, IEEE), SYAFRUDDIN SYARIF[1], AND MUHAMMAD NISWAR[2]
[1]Department of Electrical Engineering, Hasanuddin University, Makassar, Indonesia
[2]Department of Informatics, Hasanuddin University, Makassar, Indonesia
[3]Department of Informatics, Faculty of Computer Science, Ichsan Sidenreng Rappang University, Sidenreng Rappang, Indonesia

Corresponding author: First A. Author (e-mail: author@ boulder.nist.gov).

This paragraph of the first footnote will contain support information, including sponsor and financial support acknowledgment. For example, "This work was supported in part by the U.S. Department of Commerce under Grant BS123456."

ABSTRACT These instructions give you guidelines for preparing papers for IEEE Access. Use this document as a template if you are using LaTeX. Otherwise, use this document as an instruction set. The electronic file of your paper will be formatted further at IEEE. Paper titles should be written in uppercase and lowercase letters, not all uppercase. Avoid writing long formulas with subscripts in the title; short formulas that identify the elements are fine (e.g., "Nd–Fe–B"). Do not write "(Invited)" in the title. Full names of authors are preferred in the author field, but are not required. Put a space between authors' initials. The abstract must be a concise yet comprehensive reflection of what is in your article. In particular, the abstract must be self-contained, without abbreviations, footnotes, or references. It should be a microcosm of the full article. The abstract must be between 150–250 words. Be sure that you adhere to these limits; otherwise, you will need to edit your abstract accordingly. The abstract must be written as one paragraph, and should not contain displayed mathematical equations or tabular material. The abstract should include three or four different keywords or phrases, as this will help readers to find it. It is important to avoid over-repetition of such phrases as this can result in a page being rejected by search engines. Ensure that your abstract reads well and is grammatically correct.

INDEX TERMS Enter key words or phrases in alphabetical order, separated by commas. Autocorrelation, beamforming, communications technology, dictionary learning, feedback, fMRI, mmWave, multipath, system design, multipath, slight fault, underlubrication fault.

I. INTRODUCTION

Edge caching is a fundamental mechanism for reducing user-perceived latency and alleviating upstream bandwidth consumption in content delivery networks and edge computing environments. By storing a subset of objects near end users, caches can exploit temporal locality and reduce redundant transfers over backhaul links, which is particularly valuable for delay-sensitive services [1]. However, practical request streams exhibit non-stationary content popularity, where the identity and intensity of popular objects vary over time rather than remaining stable across long horizons [2], [3]. Static caching solutions may fail to track evolving demand, lowering hit ratio and increasing service latency [4]. This motivates adaptive caching policies that can respond to changing request patterns and sustain performance over time [5].

Recent literature therefore emphasizes proactive and predictive caching as a practical response to time-varying demand, often combining proactive placement with reactive serving [6]. Nevertheless, the central challenge in this setting is cache management under concept drift, where the underlying popularity distribution shifts due to changes in user behavior, trends, and events, and previously learned predictors become quickly outdated [7], [8]. Predictive components rely on forecasting or learning models, ranging from classical time-series methods such as ARIMA to modern deep models [9]. When forecasts are accurate, prediction can improve hit ratio and reduce network traffic, particularly in edge settings where mobility and context affect demand [10], [11]. Yet robust operation requires continual adaptation so that cache refresh remains aligned with evolving popularity rather than frozen on historical evidence [12].

Specific solutions in the literature implement adaptation through reinforcement learning, online or incremental learning, and drift detection. Reinforcement learning approaches, including deep reinforcement learning, learn caching decisions from interaction and can adapt policies under changing demand [13]. Multi-agent reinforcement learning further targets distributed environments, where coordinated decisions

can improve responsiveness to fluctuating content scenarios [3]. In parallel, learning-based popularity prediction improves caching by combining historical and recent signals [11], [14]. To address non-stationarity without repeated full retraining, online and incremental learning methods update models continuously, often using ensembles to balance stability and plasticity [15], [16]. Drift detection frameworks complement these methods by monitoring distributional discrepancies and triggering adaptation when shifts are detected [17], [18].

A particularly relevant prior solution is incremental learning-based edge caching for non-stationary workloads. Xu et al. propose an incremental learning-based caching system that preserves useful knowledge while adapting to dynamic workloads, and this family often draws on Learn++.NSE to learn under non-stationary conditions [19], [20]. However, proactive admission introduces a systems risk that predictive accuracy alone does not resolve: incorrect admissions can cause cache pollution and churn by evicting useful objects and occupying space with low-utility content [21], [22]. Prior work therefore stresses admission control and feedback mechanisms that constrain unnecessary insertions [23]. Separately, drift metrics such as Jensen-Shannon Divergence and set-overlap measures have been used to monitor distribution shifts [24], [25], but they are often not mapped into explicit cache refresh pressure that is online-safe and aligned with admission delay. Motivated by this gap, we couple drift monitoring, proactive admission control, and cache-aware guardrails in a unified design.

The main contributions of this paper are summarized as follows:

1) Drift-aware proactive admission control. We map a divergence-based drift signal (JSD with EMA-smoothed weighted Top-K overlap) into cache refresh pressure for Top-M admission with one-slot delayed application $(t + 1)$, while keeping replacement LRU unchanged and decisions online-safe.
2) Cache-aware guardrails. We introduce lag-aware precision and score-quality gating to stabilize proactive admission and mitigate churn and cache pollution when ranking separation is weak under non-stationary demand.
3) Online-safe features and slot-level diagnostics. We propose multi-timescale online-safe features (gap/recency plus multi-scale frequency) and conduct slot-by-slot diagnostics to audit the relationship among drift, admission behavior, and hit ratio across datasets and cache sizes.

The remainder of this paper is organized as follows. Section II reviews related work on proactive caching, learning-based admission, and learning under drift. Section III summarizes the learning and drift concepts required by the proposed design. Section IV formalizes the system model and problem formulation. Section V presents the proposed drift-aware proactive caching method, including features, labeling, incremental learning, and guardrails. Section VI reports trace-driven evaluations across cache sizes and datasets. Section VII discusses diagnostic findings, limitations, and implications for deployment. Section VIII concludes the paper and outlines future directions.

II. RELATED WORK

Proactive and predictive caching has been widely studied to improve cache performance by pre-positioning content ahead of demand peaks, rather than relying on reactive fills after misses. Taxonomies of edge and extreme-edge computing highlight how differences in topology, device capability, and coordination constraints shape caching and resource-management choices [26]. Surveys of online data-driven proactive techniques further document the increasing integration of learning components into caching and related control tasks, especially in mobile edge settings [27]. A recurring architectural pattern combines proactive placement with reactive serving, as in hybrid proactive-reactive policies that balance anticipation and responsiveness [6]. Methodologically, predictive caching spans classical forecasting such as ARIMA and learning-based popularity prediction using machine learning or deep models [9]. Recent work also incorporates contextual and mobility signals, including federated learning and spatiotemporal modeling, to better align placement with evolving demand [11], [28].

Despite this breadth, the primary difficulty is that content popularity is rarely stationary. Request streams often exhibit time-varying demand in which both the identity and intensity of popular items evolve, so policies anchored in long-horizon statistics can become misaligned with current requests [2], [3]. This misalignment has direct implications for hit ratio and user latency because stale cache contents reduce the fraction of requests served locally, even when cache capacity is adequate [4]. Concept drift formalizes the problem as changes in the underlying request distribution driven by shifts in user behavior, events, and emerging trends [7], [8]. Under drift, predictive models that previously performed well can degrade quickly unless they are updated at a pace comparable to the change in the stream, which motivates continual adaptation rather than periodic retraining alone [12].

To cope with drift, adaptive approaches update predictors and cache decisions continuously. Reinforcement learning, including deep reinforcement learning, learns admission, placement, or eviction strategies from interaction and can respond to changing request patterns [13], while multi-agent reinforcement learning targets distributed environments where coordinated decisions may be beneficial [3]. In parallel, online and incremental learning updates popularity predictors without full retraining, and ensemble methods are frequently emphasized because they can balance stability and plasticity by retaining useful historical hypotheses while adapting to recent data [15], [16]. Drift handling is also discussed as active versus passive adaptation, where active methods employ explicit drift detection to trigger updates [29]. Drift detection frameworks monitor distributional discrepancies over time and provide signals that can be used to schedule model refresh

or other system actions [17], [18].

Closest to the present study are incremental learning-based caching systems for non-stationary workloads. Xu et al. propose an incremental learning-based edge caching system that preserves useful knowledge while adapting to workload dynamics, and this line often relies on Learn++.NSE as an ensemble learner for non-stationary environments [19], [20]. However, proactive caching and prefetching remain sensitive to errors in prediction and admission, which can induce cache pollution and churn by evicting useful items and filling the cache with low-utility content [21], [22]. This risk motivates admission control and filtering policies that constrain insertions, including tuned admission parameters and frequency-based mechanisms such as TinyLFU [23], [30], [31]. Accordingly, evaluation practice increasingly recommends analyzing hit ratio alongside eviction and churn indicators, since aggregate hit ratio can mask instability caused by excessive turnover [32].

Taken together, the literature supports proactive placement and continual adaptation under dynamic popularity, yet it leaves a systems-level gap between drift monitoring, learning outputs, and cache control. Although drift detection and incremental learning mechanisms are available, it is often unclear how drift signals should be mapped into cache refresh pressure under an admission timeline and a fixed replacement policy, especially when proactive decisions are applied with delay and produce delayed utility [12]. Moreover, while pollution-aware admission control is well studied, these safeguards are rarely coupled to drift severity and to measures of ranking confidence that indicate when aggressive refresh is likely to be harmful [21]. Finally, trace-driven protocols emphasize time-slotting and window-based analysis to connect evolving demand to cache dynamics rather than relying on aggregate averages [33]–[35]. This study positions itself in this gap by coupling drift monitoring with proactive admission control and cache-aware guardrails, and by using slot-level diagnostics to relate drift behavior, admission decisions, and hit-ratio outcomes.

III. PRELIMINARIES

A. SYSTEM AND NOTATION

We consider a request stream $\{r_i\}_{i=1}^{N}$ arriving at an edge cache. Each request r_i refers to an object $o(r_i)$ in a finite universe O. The cache stores at most C objects (or C units in a normalized object-size setting) and serves requests either as cache hits or misses. A hit occurs when the requested object is already stored; otherwise a miss triggers retrieval from an origin server and may lead to insertion depending on the admission policy. The cache hit ratio (HR) is defined as the fraction of requests served as hits over an evaluation interval, and it is a primary metric for evaluating caching strategies in request streams [36], [37]. Throughout the paper, we report HR as an empirical average over the trace and compare it across different cache capacities.

When an insertion occurs and the cache is full, a replacement policy selects a victim object to evict. Least Recently Used (LRU) is a common baseline because it exploits temporal locality with low overhead, and variants of LRU remain actively studied for improving hit ratios under diverse workloads [38], [39]. Since HR depends on both capacity and management policy, we evaluate each method under multiple cache sizes, reflecting the well-known sensitivity of hit probability to the relationship between C and the workload's effective working set [40]. To avoid ambiguity, we distinguish replacement, which decides which resident object is removed upon insertion, from admission, which decides whether a fetched object becomes eligible for insertion.

B. TIME-SLOTTED TIMELINE AND ONLINE-SAFETY

To support proactive decisions under non-stationary demand, we partition the request stream into contiguous time slots indexed by t. All requests arriving within a slot form the observable data for that slot, including per-object request counts, hit/miss events, and recency updates. The system computes statistics and features during slot t, then finalizes learning and admission decisions at the slot boundary using information available up to that time. This time-window abstraction is widely used to model time-varying popularity and to schedule proactive caching actions [41]. Our notion of online safety follows the standard requirement that decisions are causal, in the sense that future requests beyond the current slot are not used, which aligns with online caching settings where the algorithm cannot rely on knowledge of future requests [35].

In our design, objects selected for admission at the end of slot t are inserted at the beginning of slot $t + 1$. This one-slot delay avoids any dependence on within-slot future observations when determining the final admitted set, and it reflects causal update schemes that rely on data from the previous slot rather than future information. The delay also introduces a delayed utility effect: an object that would have been useful later in slot t cannot contribute hits until slot $t + 1$, and aggressive refresh can therefore increase churn without immediate benefit. Prior work on proactive caching emphasizes that admission control must account for such timing and resource constraints to maintain efficiency under real-time operation [42], [43].

C. INCREMENTAL LEARNING UNDER DRIFT (LEARN++.NSE)

Concept drift denotes changes in the data-generating process over time, which can cause predictors trained on earlier data to become miscalibrated on later samples. In non-stationary streams, this motivates incremental learning, where a model is updated sequentially as new labeled data become available, instead of being retrained from scratch. Such sequential learning is attractive when both latency and computational cost matter, and it has been studied as a principled response to drift in streaming settings [12]. Drift handling is commonly coupled with monitoring mechanisms that detect distributional changes or performance degradation and guide when and how to adapt the model [44]. In caching, incremental learning has

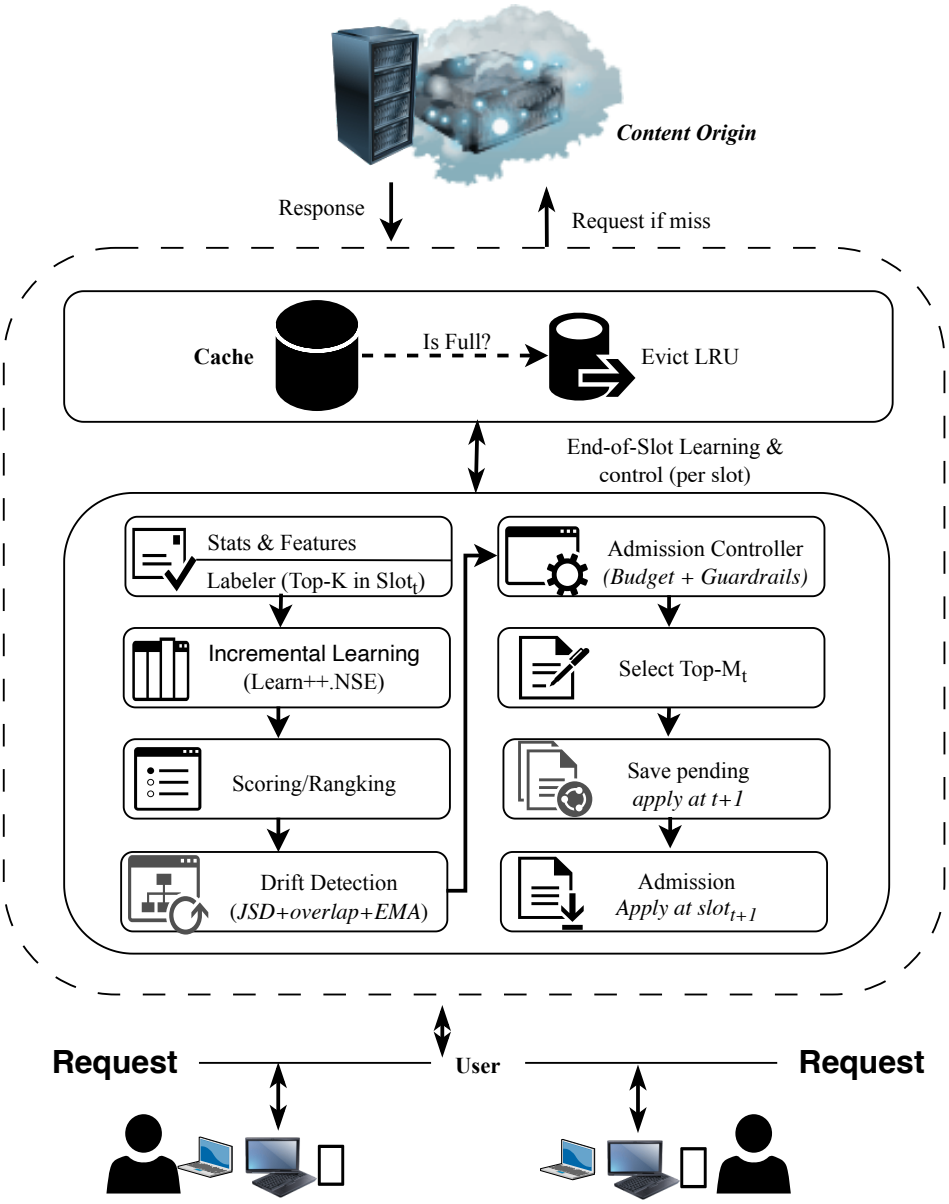

FIGURE 1. **Magnetization as a function of applied field. It is good practice to explain the significance of the figure in the caption.**

been reported to improve hit ratio by updating popularity predictors to track evolving workloads [19].

Learn++.NSE is an incremental ensemble method designed for non-stationary environments. It trains a sequence of base classifiers on successive data batches and forms an ensemble whose members are weighted according to their relevance to the current concept [20]. By reweighting rather than discarding all prior hypotheses, the method aims to balance plasticity, which is needed to learn new concepts, and stability, which preserves useful historical knowledge. Ensemble learning is a natural fit for this objective because different learners can specialize to different regions or phases of the data stream, improving robustness under distribution shift [45]. In our setting, each time slot provides a natural batch for model update, which makes Learn++.NSE compatible with per-slot retraining constraints while retaining the ability to react to drift.

Implementations of incremental learning in streaming systems often combine ensemble updating with mechanisms that emphasize recent evidence and suppress outdated components, sometimes guided by drift detection or online model selection [15], [46]. We follow this general perspective by updating the learning component at slot boundaries using labels derived from within-slot observations and features that are computable without future information. Although the detailed feature construction and admission control are introduced

in Section V, the key preliminary point is that the learning update is synchronized with the cache control cycle. This synchronization ensures that model adaptation and caching actions evolve on the same time scale, which is critical when non-stationary popularity changes at a pace comparable to the slot duration.

IV. SYSTEM MODEL AND PROBLEM FORMULATION

A. SYSTEM ARCHITECTURE AND OPERATION TIMELINE

We consider a single edge node equipped with a cache that serves end-user requests and fetches missed objects from an origin repository, consistent with common edge caching deployments aimed at reducing delivery latency and upstream traffic [47]. The cache is augmented with an admission control module, which is broadly motivated by evidence that selective admission can improve performance under dynamic demand compared with purely reactive insertion [43]. Fig. 1 summarizes the operational pipeline: requests are served online, while statistics and learning components run at slot boundaries to produce admission decisions. This separation supports incremental adaptation to non-stationary popularity without requiring the replacement policy to change [19].

Formally, the request stream is indexed by i and time is partitioned into slots indexed by t. Each request is

$$r_i = (o_i, \tau_i), \qquad R_t = \{\, r_i : \tau_i \in \text{slot } t \,\}, \tag{1}$$

where $o_i \in U$ is an object from universe U. At the end of slot t, the controller selects an admission set A_t using only information observed up to that boundary, and applies it at the beginning of slot $t+1$:

$$A_t = \pi(\mathcal{F}_t), \qquad \mathcal{F}_t \text{ uses only data up to the end of slot } t, \tag{2}$$

which matches the causal requirement of online caching and time-window decision making [35].

B. CACHE DYNAMICS AND ASSUMPTIONS

Let $S_t \subseteq U$ denote the cache state during slot t, constrained by capacity C:

$$|S_t| \le C. \tag{3}$$

For request r_i, the hit indicator is

$$h_i = 1\big[\, o_i \in S_{t(i)} \,\big], \tag{4}$$

where $t(i)$ is the slot index containing τ_i. We evaluate the overall hit ratio over the full trace $R = \bigcup_t R_t$ as

$$\mathrm{HR} = \frac{1}{|R|} \sum_{r_i \in R} h_i \tag{5}$$

following standard definitions used to model cache performance and hit rates in request streams [36], [48].

We separate admission from replacement. Admissions determine which objects become candidates for insertion, while replacement selects evictions when the cache is full. We keep replacement fixed to LRU, a widely used baseline that exploits temporal locality with low management overhead [49]. Given the delayed application, the cache state update at the slot boundary is written abstractly as

$$S_{t+1} = \mathrm{LRU}_C(S_t \cup A_t)\,, \tag{6}$$

meaning that objects in A_t are inserted and LRU evicts as needed to satisfy (3) [49]. Under time-varying popularity, inaccurate admissions can increase churn and indirectly reduce hit ratio, which motivates coupling admission policies to signals that reflect workload change rather than assuming stationarity [41].

C. OBJECTIVE AND CONSTRAINTS

For a given capacity C, our objective is to find a policy π that maximizes hit ratio under online operation:

$$\max_{\pi}\ \mathrm{HR}(C) \quad \text{s.t. } (2),\ (3),\ \text{and } (6). \tag{7}$$

This objective aligns with the common goal of improving cache hit performance to reduce server load and delivery delay in caching systems [50]. The online-safe constraint (2) is essential because caching decisions must be made without access to future requests, which is a defining characteristic of online learning and streaming operation [12], [35]. The one-slot application delay in (6) induces delayed utility, since admitted objects cannot produce hits until slot $t+1$, and this timing must be accounted for in the policy design.

We further constrain admissions by a per-slot budget M_t determined by π:

$$A_t \subseteq U, \qquad |A_t| \le M_t, \tag{8}$$

which captures practical limits on how aggressively the cache can be refreshed [43]. We treat computational and operational overhead as a design constraint described narratively, since the paper's evaluation emphasizes hit ratio and stability rather than deriving a separate overhead objective; nonetheless, the policy is intended to remain feasible for edge settings where excessive control cost is undesirable. The specific construction of π, including how it sets M_t and how it uses learning and drift signals, is presented in Section V.

V. PROPOSED DRIFT-AWARE PROACTIVE ADMISSION METHOD

This section presents a drift-aware proactive admission method that improves cache refresh under nonstationary demand while preserving online safety. The design keeps the replacement policy fixed as LRU and concentrates learning and control at the admission stage, so performance changes can be attributed to candidate selection and admission pressure rather than to a new eviction rule [19]. Fig. 2 summarizes the online path and end-of-slot control loop, including the one-slot delayed application of admissions.

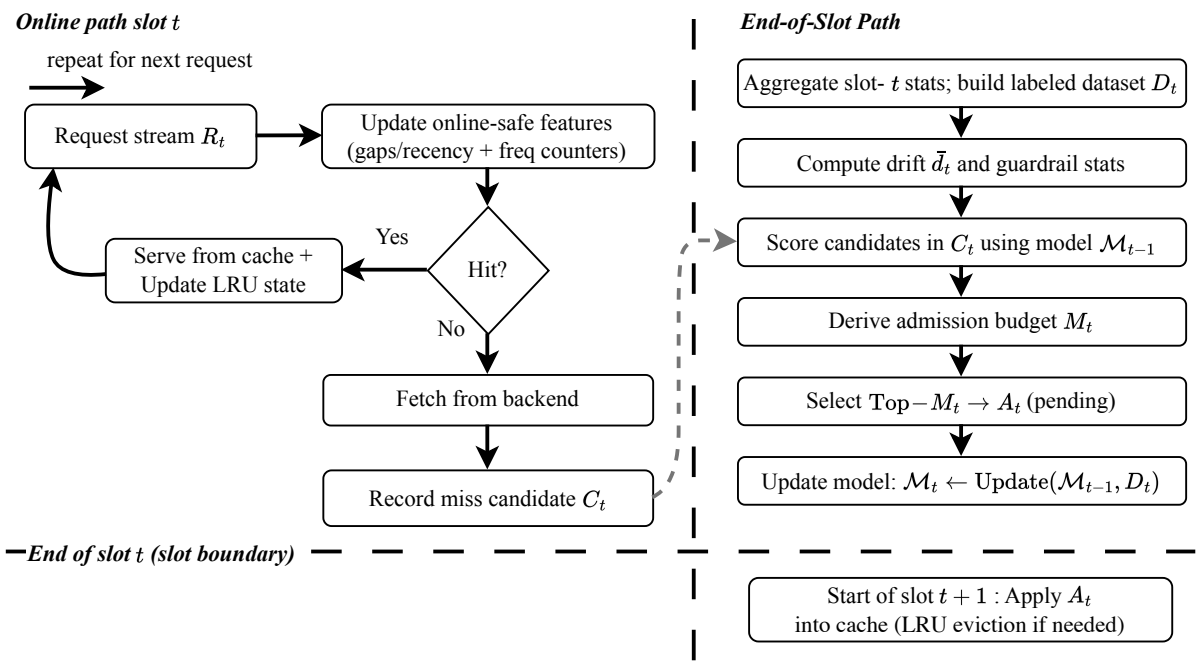

FIGURE 2. **Magnetization as a function of applied field. It is good practice to explain the significance of the figure in the caption.**

A. OVERVIEW AND EXECUTION ORDER

Within slot t, the cache serves the request stream $\mathcal{R}_t$ and updates LRU state on hits, while misses are fetched from the backend and recorded as admission candidates $\mathcal{C}_t$ [35]. At the slot boundary, the system aggregates slot statistics and constructs a labeled dataset D_t, computes drift and guardrail statistics, and scores candidates using the model state from the previous slot. This causal execution order follows the "decide first, then update" principle shown in Fig. 2.

Formally, candidates are scored with $\mathcal{M}_{t-1}$, then a pending admission set A_t is selected under a budget M_t, and only afterward the incremental learner is updated so that $\mathcal{M}_t$ is used starting in slot $t+1$ [19], [20]. The delayed application is captured by A_t being inserted at the start of slot $t+1$, consistent with online decision making without future requests [35].

B. ONLINE-SAFE FEATURES AND SLOT-LEVEL LABELING

Each object $o \in \mathcal{U}$ is represented at the end of slot t by an online-safe feature vector $X_t(o)$ computed only from information observed up to the slot boundary. We use (i) recency and gap statistics to exploit temporal locality and (ii) multi-timescale frequency counters to summarize evolving popularity [1], [2]. Let $last_t(o)$ be the most recent slot index where o appeared up to slot t. The slot-level recency gap is

$$g_t(o) = t - \mathrm{last}_t(o), \tag{9}$$

which is maintained causally from the request stream [1].

In addition, we maintain frequency counters $f_t^{(w)}(o)$ for multiple windows $w \in \{\mathrm{short}, \mathrm{mid}, \mathrm{long}, \mathrm{cum}\}$, updated online from requests, to reflect both short-term bursts and longer-term reuse [2]. Slot-level labeling follows the $Top-K$ scheme used in incremental-learning caching pipelines [19]. Let $\mathcal{C}_t(o)$ be the request count of o in slot t, and let $TopK_t$ be the set of the most requested objects in slot t, with K chosen as a fixed fraction (e.g., 20%) of the distinct objects in that slot [19]. The label is

$$y_t(o) = 1\big[\, o \in \mathrm{TopK}_t \,\big], \tag{10}$$

computed after slot t completes and used only at the boundary, which preserves online safety under slotting [35].

C. INCREMENTAL UPDATE AND CANDIDATE SCORING WITH LEARN++.NSE

We adopt Learn++.NSE as the incremental learner because it is designed for nonstationary environments using an ensemble whose contribution can shift toward more recent learners while retaining useful historical hypotheses [20]. This aligns with caching workloads where the popularity distribution can drift and fixed predictors can become stale if not refreshed incrementally [12]. In each slot, the training set is $D_t = \{(x_t(o), y_t(o))\}$, and the model update is executed after admission selection, so the updated model is not used to score candidates that were evaluated using labels from the same slot [19].

Candidate scoring produces a real-valued score used for ranking miss candidates in $\mathcal{C}_t$. For each $o \in \mathcal{C}_t$,

$$s_t(o) = \mathcal{M}_{t-1}\big(x_t(o)\big), \tag{11}$$

where $s_t(o)$ can be interpreted as the ensemble's confidence for the positive class under Learn++.NSE [20]. After selecting

A_t, the model is updated as $\mathcal{M}_t \leftarrow \mathrm{Update}(\mathcal{M}_{t-1}, D_t)$, so $\mathcal{M}_t$ is used starting in slot $t+1$ [19].

D. DRIFT SIGNAL AND DRIFT-AWARE ADMISSION CONTROLLER

To adapt admission pressure to distribution shifts, we compute a drift signal from the popularity observed in recent slots. Let P_t be a discrete distribution over $TopK_t$, obtained by normalizing within-slot counts $\mathcal{C}_t(o)$ over that set. We quantify changes between adjacent slots using Jensen–Shannon divergence,

$$\mathrm{JSD}_t = \mathrm{JSD}(P_t \,\|\, P_{t-1}), \tag{12}$$

which is a common divergence metric for monitoring distribution change [24], [25]. Since caching performance also depends on the identity of top items, we incorporate a rank-weighted overlap term over $\mathrm{TopK}_t \cap \mathrm{TopK}_{t-1}$ to penalize rapid turnover of highly ranked objects [25].

We combine divergence and overlap into a raw drift score and smooth it to avoid overreacting to transient noise:

$$d_t = \lambda\,\mathrm{JSD}_t + (1-\lambda)(1-\mathrm{WO}_t), \qquad \bar{d}_t = \beta\,\bar{d}_{t-1} + (1-\beta)d_t, \tag{13}$$

where WO_t denotes the rank-weighted overlap and $\bar{d}_t$ is an EMA-smoothed drift signal [25]. The controller maps $\bar{d}_t$ to an admission budget $M_t \in [M_{\min}, M_{\max}]$ and selects the pending admission set A_t by taking the top-ranked candidates under $s_t(o)$ subject to $|A_t| \le M_t$, using only information available up to slot t [35].

E. CACHE-AWARE GUARDRAILS AND COMPUTATIONAL

Proactive admission can improve hit ratio when predictions are accurate, but incorrect admissions can increase pollution and churn by displacing useful objects, degrading cache efficiency [21], [22]. To address delayed utility under one-slot admission, we use lag-aware precision feedback. For lag $\ell \in \{0,1,2\}$, let $\mathrm{Hit}_t(A_{t-\ell})$ be the number of requests in slot t served by objects admitted at slot $(t-\ell$. We define

$$\mathrm{Prec}_t^{(\ell)} = \frac{\mathrm{Hit}_t(A_{t-\ell})}{\max(1, |A_{t-\ell}|)}, \qquad \mathrm{Prec}_t^{\mathrm{eff}} = \sum_{\ell} \omega_\ell \,\mathrm{Prec}_t^{(\ell)}, \tag{14}$$

and reduce admission pressure when $\mathrm{Prec}_t^{\mathrm{eff}}$ indicates weak realized utility, consistent with the role of tuned admission in limiting unnecessary insertions [23].

We further apply score-quality gating to reduce M_t when the score separation among top candidates is small, since weak ranking separation is a common failure mode that can trigger churn under proactive insertion [21]. In terms of cost, feature maintenance is $O(|\mathcal{R}_t|)$ per slot, scoring is $O(|C_t|\kappa)$ where κ is per-candidate inference cost, and selection can be implemented via partial ranking for $top - M_t$. Learn++.NSE update cost is controlled by bounding the number of retained base learners, consistent with ensemble practice in nonstationary settings [20]. This bounded per-slot workload matches the practical need to limit operational overhead in constrained caching systems.

ACKNOWLEDGMENT

The preferred spelling of the word "acknowledgment" in American English is without an "e" after the "g." Use the singular heading even if you have many acknowledgments. Avoid expressions such as "One of us (S.B.A.) would like to thank . . . ." Instead, write "F. A. Author thanks . . . ." In most cases, sponsor and financial support acknowledgments are placed in the unnumbered footnote on the first page, not here.

REFERENCES

[1] A. Sadeghi, F. Sheikholeslami, A. G. Marqués, and G. B. Giannakis, "Reinforcement Learning for Adaptive Caching With Dynamic Storage Pricing," *Ieee Journal on Selected Areas in Communications*, vol. 37, no. 10, pp. 2267–2281, 2019.

[2] P. Yang, N. Zhang, S. Zhang, L. Yu, J. Zhang, and X. Shen, "Content Popularity Prediction Towards Location-Aware Mobile Edge Caching," *Ieee Transactions on Multimedia*, vol. 21, no. 4, pp. 915–929, 2019.

[3] B. Guo, Y. Chen, P. Cheng, M. Ding, J. Hu, and L. Hanzo, "Pareto-Optimal Multiagent Cooperative Caching Relying on Multipolicy Reinforcement Learning," *Ieee Internet of Things Journal*, vol. 11, no. 5, pp. 7904–7917, 2024.

[4] B. Jedari and M. Di Francesco, "Auction-based cache trading for scalable videos in multi-provider heterogeneous networks," in *Proceedings of the IEEE Conference on Computer Communications (INFOCOM)*, pp. 1864–1872, 2019.

[5] R. Bhattacharjee, S. Banerjee, and A. Sinha, "Fundamental Limits on the Regret of Online Network-Caching," *Acm Sigmetrics Performance Evaluation Review*, vol. 48, no. 1, pp. 15–16, 2020.

[6] K. Qi, S. Han, and C. Yang, "Learning a Hybrid Proactive and Reactive Caching Policy in Wireless Edge Under Dynamic Popularity," *IEEE Access*, vol. 7, pp. 120788–120801, 2019.

[7] Z. Yang, Y. Liu, Y. Chen, and J. T. Zhou, "Deep Learning for Latent Events Forecasting in Content Caching Networks," *Ieee Transactions on Wireless Communications*, vol. 21, no. 1, pp. 413–428, 2022.

[8] X. Zhang, G. Zheng, S. Lambotharan, M. R. Nakhai, and K.-K. Wong, "A learning approach to edge caching with dynamic content library in wireless networks," in *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*, 2019.

[9] K. T. M. Abarna and T. Suresh, "Enrich Multi-Channel P2P VoD Streaming Based on Dynamic Replication Strategy," *International Journal of Advances in Applied Sciences*, vol. 9, no. 2, p. 110, 2020.

[10] S. He, J. Ren, J. Wang, Y. Huang, Y. Zhang, W. Zhuang, and X. Shen, "Cloud-Edge Coordinated Processing: Low-Latency Multicasting Transmission," *Ieee Journal on Selected Areas in Communications*, vol. 37, no. 5, pp. 1144–1158, 2019.

[11] Z. Yu, J. Hu, G. Min, Z. Zhao, W. Miao, and M. S. Hossain, "Mobility-Aware Proactive Edge Caching for Connected Vehicles Using Federated Learning," *Ieee Transactions on Intelligent Transportation Systems*, vol. 22, no. 8, pp. 5341–5351, 2021.

[12] P. Wu and N. Xiong, "Online Learning of Bayesian Classifiers With Nonstationary Data Streams," *International Journal of Network Dynamics and Intelligence*, p. 100009, 2023.

[13] D. Liu and C. Yang, "A Deep Reinforcement Learning Approach to Proactive Content Pushing and Recommendation for Mobile Users," *Ieee Access*, vol. 7, pp. 83120–83136, 2019.

[14] F. Lei, J. Cai, Q. Dai, and H. Zhao, "Deep Learning Based Proactive Caching for Effective WSN-Enabled Vision Applications," *Complexity*, vol. 2019, no. 1, 2019.

[15] N. Liu and Z. Jian-hua, "Streaming Data Classification Based on Hierarchical Concept Drift and Online Ensemble," *Ieee Access*, vol. 11, pp. 126040–126051, 2023.

[16] H. Du, L. L. Minku, and H. Zhou, "Multi-source transfer learning for non-stationary environments," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2019.

[17] J. Zhu, S. Cai, F. Deng, B. C. Ooi, and W. Zhang, "METER: A Dynamic Concept Adaptation Framework for Online Anomaly Detection," *Proceedings of the VLDB Endowment*, vol. 17, no. 4, pp. 794–807, 2023.

[18] A. Liu, J. Lü, and G. Zhang, "Concept Drift Detection via Equal Intensity K-Means Space Partitioning," *Ieee Transactions on Cybernetics*, vol. 51, no. 6, pp. 3198–3211, 2021.

[19] G. Xu, B. Tang, L. Yuan, Y. Xue, Z. Gao, S. Mostafa, and C. W. Sung, "An Incremental Learning Based Edge Caching System: From Modeling to Evaluation," *IEEE Access*, vol. 8, pp. 12499–12509, 2020.

[20] R. Elwell and R. Polikar, "Incremental Learning of Concept Drift in Nonstationary Environments," *IEEE Transactions on Neural Networks*, vol. 22, pp. 1517–1531, Oct. 2011.

[21] D. Deb and J. Jose, "ZPP: A Dynamic Technique to Eliminate Cache Pollution in NoC Based MPSoCs," *Acm Transactions on Embedded Computing Systems*, vol. 22, no. 5s, pp. 1–25, 2023.

[22] D. Deb, J. Jose, and M. Palesi, "ECAP: Energy-efficient Caching for Prefetch Blocks in Tiled Chip Multiprocessors," *Iet Computers & Digital Techniques*, vol. 13, no. 6, pp. 417–428, 2019.

[23] D. Man, Y. Mu, J. Guo, W. Yang, J. Lv, and W. Wang, "Cache Pollution Detection Method Based on GBDT in Information-Centric Network," *Security and Communication Networks*, vol. 2021, pp. 1–10, 2021.

[24] E. S. Andersen, J. B. Birk-Korch, R. S. Hansen, L. H. Fly, R. H. Roettger, D. M. C. Arcani, C. L. Brasen, I. Brandslund, and J. S. Madsen, "Monitoring Performance of Clinical Artificial Intelligence in Health Care: A Scoping Review," *Jbi Evidence Synthesis*, vol. 22, no. 12, pp. 2423–2446, 2024.

[25] S. Basterrech, J. Platoš, G. Rubino, and M. Woźniak, "Experimental analysis on dissimilarity metrics and sudden concept drift detection," in *Intelligent Systems Design and Applications*, pp. 190–199, Springer, 2023.

[26] R. Immonen and T. Hämäläinen, "Tiny Machine Learning for Resource-Constrained Microcontrollers," *Journal of Sensors*, vol. 2022, pp. 1–11, 2022.

[27] B. Ma, W. Guo, and J. Zhang, "A Survey of Online Data-Driven Proactive 5G Network Optimisation Using Machine Learning," *Ieee Access*, vol. 8, pp. 35606–35637, 2020.

[28] S. Mehrizi, S. Chatterjee, S. Chatzinotas, and B. Ottersten, "Online Spatiotemporal Popularity Learning via Variational Bayes for Cooperative Caching," *Ieee Transactions on Communications*, vol. 68, no. 11, pp. 7068–7082, 2020.

[29] M. Han, Z. Chen, M. Li, H. Wu, and X. Zhang, "A Survey of Active and Passive Concept Drift Handling Methods," *Computational Intelligence*, vol. 38, no. 4, pp. 1492–1535, 2022.

[30] G. Einziger and R. Friedman, "Counting With Tinytable: Every Bit Counts!," *Ieee Access*, vol. 7, pp. 166292–166309, 2019.

[31] Y. Yoji, J. Takemasa, Y. Koizumi, and T. Hasegawa, "A Packet-Level Caching Algorithm for Mitigating Negative Effects Caused by Large Objects in ICN Networks," *Ieee Access*, vol. 8, pp. 108247–108259, 2020.

[32] A. Fedorova, K. Smith, K. Bostic, A. Gorrod, S. LoVerso, and M. J. Cahill, "Writes hurt: Lessons in cache design for optane nvram," 2022.

[33] H. Xing, J. Cui, Y. Deng, and A. Nallanathan, "Energy-efficient proactive caching for fog computing with correlated task arrivals," in *Proceedings of the IEEE International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, pp. 1–5, 2019.

[34] H. Herodotou and E. Kakoulli, "Automating Distributed Tiered Storage Management in Cluster Computing," *Proceedings of the VLDB Endowment*, vol. 13, no. 1, pp. 43–56, 2019.

[35] S. Mukhopadhyay and A. Sinha, "Online caching with optimal switching regret," 2021.

[36] S. R. Pratap, S. M. Raikar, and S. V. Bhat, "Adaptive retention and eviction for efficient caching in ai-driven systems," 2025. Research Square preprint.

[37] H. Torabi, H. Khazaei, and M. Litoiu, "A learning-based caching mechanism for edge content delivery," in *ICPE '24: Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, pp. 236–246, 2024.

[38] Q. Fan, X. Li, J. Li, Q. He, K. Wang, and J. Wen, "Pa-cache: Evolving learning-based popularity-aware content caching in edge networks," 2020.

[39] X. Chen, Q. Zhu, and Y. Hua, "Joint Optimization of Recommendation and Caching Based on User Preference Prediction," *Iet Communications*, vol. 17, no. 12, pp. 1335–1353, 2023.

[40] L. Huang, S. Cheng, Y. Guan, X. Zhang, and Z. Guo, "Consistent user-traffic allocation and load balancing in mobile edge caching," in *Proceedings of the IEEE INFOCOM Workshops*, 2020.

[41] J. Gao, S. Zhang, L. Zhao, and X. Shen, "The Design of Dynamic Probabilistic Caching With Time-Varying Content Popularity," *Ieee Transactions on Mobile Computing*, vol. 20, no. 4, pp. 1672–1684, 2021.

[42] S. Sun, W. Jiang, G. Feng, S. Qin, and Y. Yuan, "Cooperative Caching With Content Popularity Prediction for Mobile Edge Caching," *Tehnicki Vjesnik - Technical Gazette*, vol. 26, no. 2, 2019.

[43] L. Abdo, I. Ahmad, and S. Abed, "A Smart Admission Control and Cache Replacement Approach in Content Delivery Networks," *Cluster Computing*, vol. 27, no. 3, pp. 2427–2445, 2023.

[44] S. Wang and L. L. Minku, "Auc estimation and concept drift detection for imbalanced data streams with multiple classes," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2020.

[45] R. V. Kulkarni, S. Revathy, and S. Patil, "An Empirical Study of Online Learning in Non-Stationary Data Streams Using Ensemble of Ensembles," *International Journal on Advanced Science Engineering and Information Technology*, vol. 11, no. 5, p. 1801, 2021.

[46] J. Wilson, S. Chaudhury, and B. Lall, "Multi-armed Bandit Based Online Model Selection for Concept-drift Adaptation," *Expert Systems*, vol. 41, no. 9, 2024.

[47] Q. T. Ngo, K. T. Phan, W. Xiang, A. N. Mahmood, and J. Slay, "Two-Tier Cache-Aided Full-Duplex Hybrid Satellite–Terrestrial Communication Networks," *Ieee Transactions on Aerospace and Electronic Systems*, vol. 58, no. 3, pp. 1753–1765, 2022.

[48] O. K. Ugwueze, C. V. Chijindu, C. C. Udeze, A. M. Ahaneku, N. J. Eneh, O. M. Ezeja, and E. C. Anoliefo, "Modeling Cache Performance for Embedded Systems," *Bulletin of Electrical Engineering and Informatics*, vol. 10, no. 5, pp. 2910–2920, 2021.

[49] L. Li, C. F. Kwong, Q. Liu, and J. Wang, "A Smart Cache Content Update Policy Based on Deep Reinforcement Learning," *Wireless Communications and Mobile Computing*, vol. 2020, pp. 1–11, 2020.

[50] F. Qazi, O. Khalid, R. N. B. Rais, I. A. Khan, and A. u. R. Khan, "Optimal Content Caching in Content-Centric Networks," *Wireless Communications and Mobile Computing*, vol. 2019, pp. 1–15, 2019.

FIRST A. AUTHOR received the B.S. and M.S. degrees in aerospace engineering from the University of Virginia, Charlottesville, in 2001 and the Ph.D. degree in mechanical engineering from Drexel University, Philadelphia, PA, in 2008.

From 2001 to 2004, he was a Research Assistant with the Princeton Plasma Physics Laboratory. Since 2009, he has been an Assistant Professor with the Mechanical Engineering Department, Texas A&M University, College Station. He is the author of three books, more than 150 articles, and more than 70 inventions. His research interests include high-pressure and high-density nonthermal plasma discharge processes and applications, microscale plasma discharges, discharges in liquids, spectroscopic diagnostics, plasma propulsion, and innovation plasma applications. He is an Associate Editor of the journal *Earth, Moon, Planets*, and holds two patents.

Dr. Author was a recipient of the International Association of Geomagnetism and Aeronomy Young Scientist Award for Excellence in 2008, and the IEEE Electromagnetic Compatibility Society Best Symposium Paper Award in 2011.

SECOND B. AUTHOR (M'76–SM'81–F'87) and all authors may include biographies. Biographies are often not included in conference-related papers. This author became a Member (M) of IEEE in 1976, a Senior Member (SM) in 1981, and a Fellow (F) in 1987. The first paragraph may contain a place and/or date of birth (list place, then date). Next, the author's educational background is listed. The degrees should be listed with type of degree in what field, which institution, city, state, and country, and year the degree was earned. The author's major field of study should be lower-cased.

The second paragraph uses the pronoun of the person (he or she) and not the author's last name. It lists military and work experience, including summer and fellowship jobs. Job titles are capitalized. The current job must have a location; previous positions may be listed without one. Information concerning previous publications may be included. Try not to list more than three books or published articles. The format for listing publishers of a book within the biography is: title of book (publisher name, year) similar to a reference. Current and previous research interests end the paragraph.

The third paragraph begins with the author's title and last name (e.g., Dr. Smith, Prof. Jones, Mr. Kajor, Ms. Hunter). List any memberships in professional societies other than the IEEE. Finally, list any awards and work for IEEE committees and publications. If a photograph is provided, it should be of good quality, and professional-looking. Following are two examples of an author's biography.

THIRD C. AUTHOR, JR. (M'87) received the B.S. degree in mechanical engineering from National Chung Cheng University, Chiayi, Taiwan, in 2004 and the M.S. degree in mechanical engineering from National Tsing Hua University, Hsinchu, Taiwan, in 2006. He is currently pursuing the Ph.D. degree in mechanical engineering at Texas A&M University, College Station, TX, USA.

From 2008 to 2009, he was a Research Assistant with the Institute of Physics, Academia Sinica, Tapei, Taiwan. His research interest includes the development of surface processing and biological/medical treatment techniques using nonthermal atmospheric pressure plasmas, fundamental study of plasma sources, and fabrication of micro- or nanostructured surfaces.

Mr. Author's awards and honors include the Frew Fellowship (Australian Academy of Science), the I. I. Rabi Prize (APS), the European Frequency and Time Forum Award, the Carl Zeiss Research Award, the William F. Meggers Award and the Adolph Lomb Medal (OSA).